

SQL JOIN – Multi-Level Lab

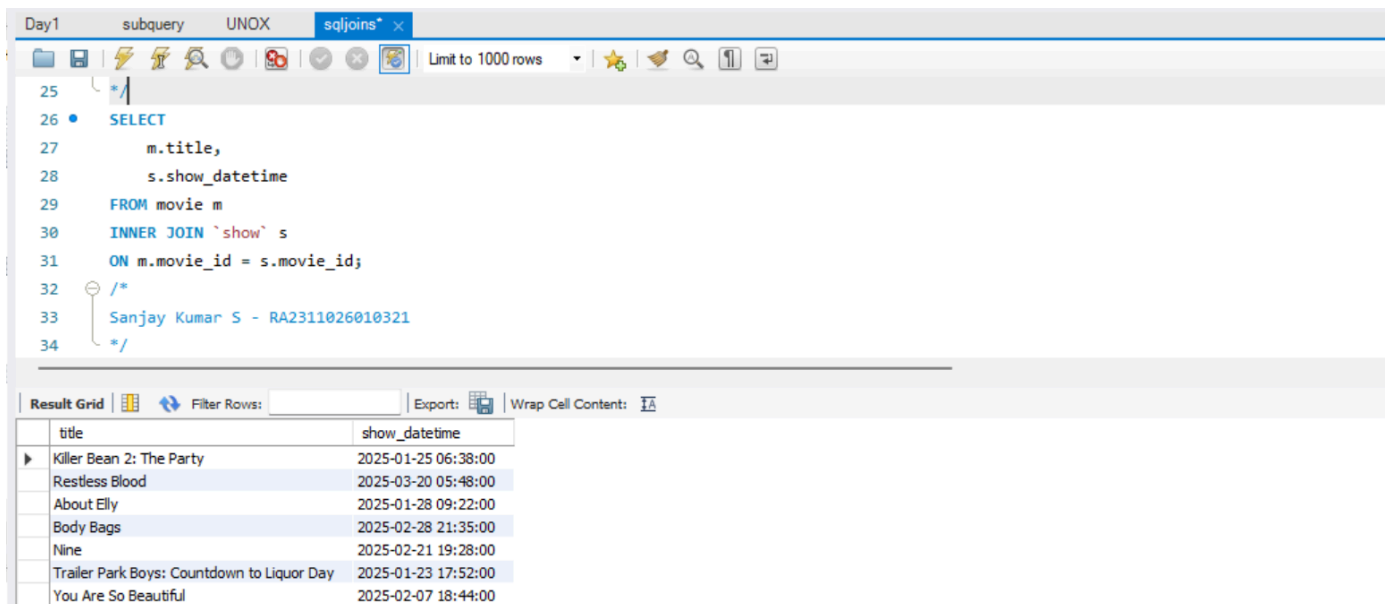
LEVEL 1 – Introduction to Joins & INNER JOIN

Problem 1: List movies and their shows

Complexity: **Easy**

Scenario:

The manager wants a list of all movies along with their scheduled show times.



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is an INNER JOIN between the 'movie' and 'show' tables. The results grid displays the following data:

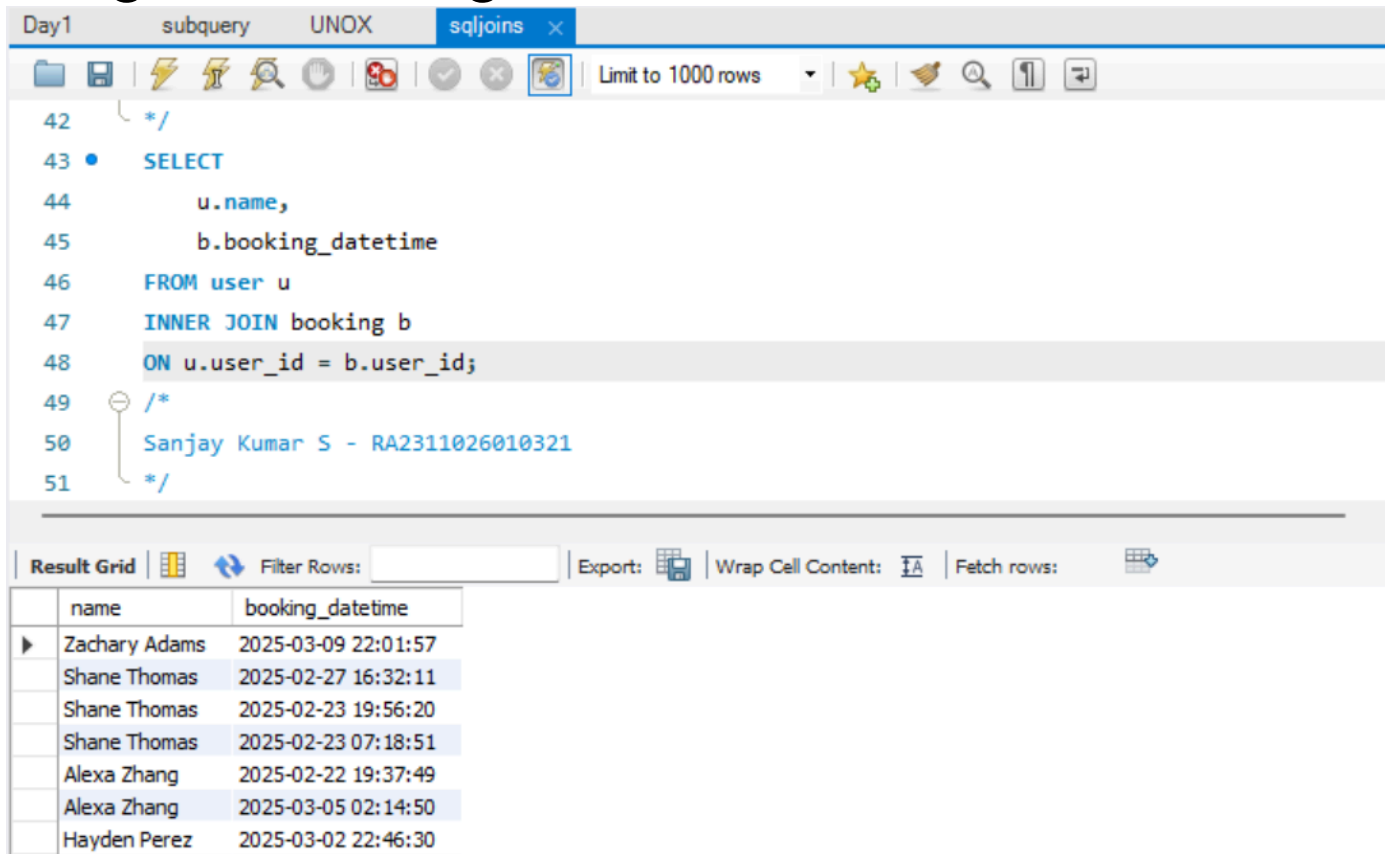
title	show_datetime
Killer Bean 2: The Party	2025-01-25 06:38:00
Restless Blood	2025-03-20 05:48:00
About Elly	2025-01-28 09:22:00
Body Bags	2025-02-28 21:35:00
Nine	2025-02-21 19:28:00
Trailer Park Boys: Countdown to Liquor Day	2025-01-23 17:52:00
You Are So Beautiful	2025-02-07 18:44:00

Problem 2: List users and their bookings

Complexity: **Easy**

Scenario:

Find all users who have made at least one booking, along with booking date.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
42  */
43  •  SELECT
44      u.name,
45      b.booking_datetime
46  FROM user u
47  INNER JOIN booking b
48  ON u.user_id = b.user_id;
49  /*
50  Sanjay Kumar S - RA2311026010321
51  */
```

The result grid displays the following data:

	name	booking_datetime
▶	Zachary Adams	2025-03-09 22:01:57
	Shane Thomas	2025-02-27 16:32:11
	Shane Thomas	2025-02-23 19:56:20
	Shane Thomas	2025-02-23 07:18:51
	Alexa Zhang	2025-02-22 19:37:49
	Alexa Zhang	2025-03-05 02:14:50
	Hayden Perez	2025-03-02 22:46:30

Problem 3 (Higher-order): Movies with more than one show

Complexity: **Medium**

Scenario:

Identify movies that have multiple shows scheduled.

The screenshot shows a SQL query editor with a query to find movies that have more than one show scheduled. The query is as follows:

```

56 scheduled.
57 =====
58 */
59 • SELECT m.movie_id,m.title,COUNT(*) AS number_of_shows FROM movie m INNER JOIN `show` s ON m.movie_id = s.movie_id GROUP BY m.movie_id, m.title HAVING COUNT(*) > 1;
60 /*
61 Sanjay Kumar S - RA2311026010321
62 */
63 /*
64 =====
65 LEVEL 2 - LEFT JOIN & RIGHT JOIN
  
```

The result grid shows the following data:

movie_id	title	number_of_shows
4	The Officers' Ward	2
5	You Are So Beautiful	2
11	Too Shy to Try	2
14	Guru	2
17	In a Year with 13 Moons	2
41	Clash of the Wolves	3
48	Rocker	2

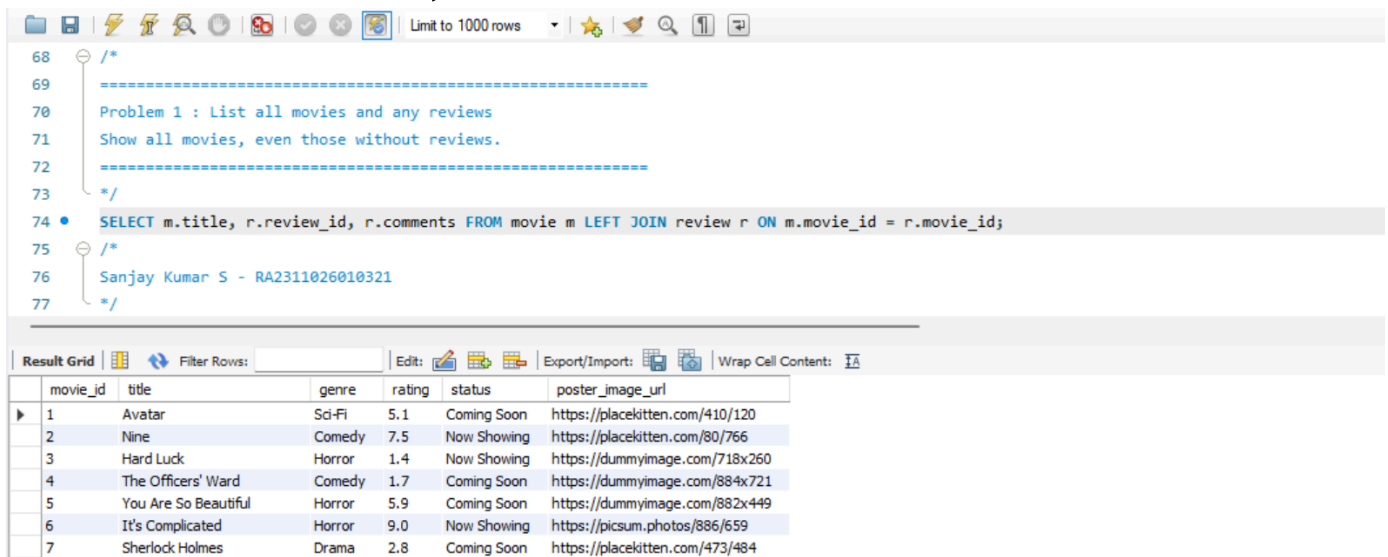
LEVEL 2 – LEFT JOIN & RIGHT JOIN

Problem 1: List all movies and any reviews

Complexity: **Easy**

Scenario:

Show all movies, even those without reviews.



The screenshot shows a SQL IDE interface. The query editor contains the following SQL query:

```

68  /*
69  =====
70  Problem 1 : List all movies and any reviews
71  Show all movies, even those without reviews.
72  =====
73  */
74  • SELECT m.title, r.review_id, r.comments FROM movie m LEFT JOIN review r ON m.movie_id = r.movie_id;
75  /*
76  Sanjay Kumar S - RA2311026010321
77  */
  
```

Below the query editor, the 'Result Grid' tab is active, displaying the following data:

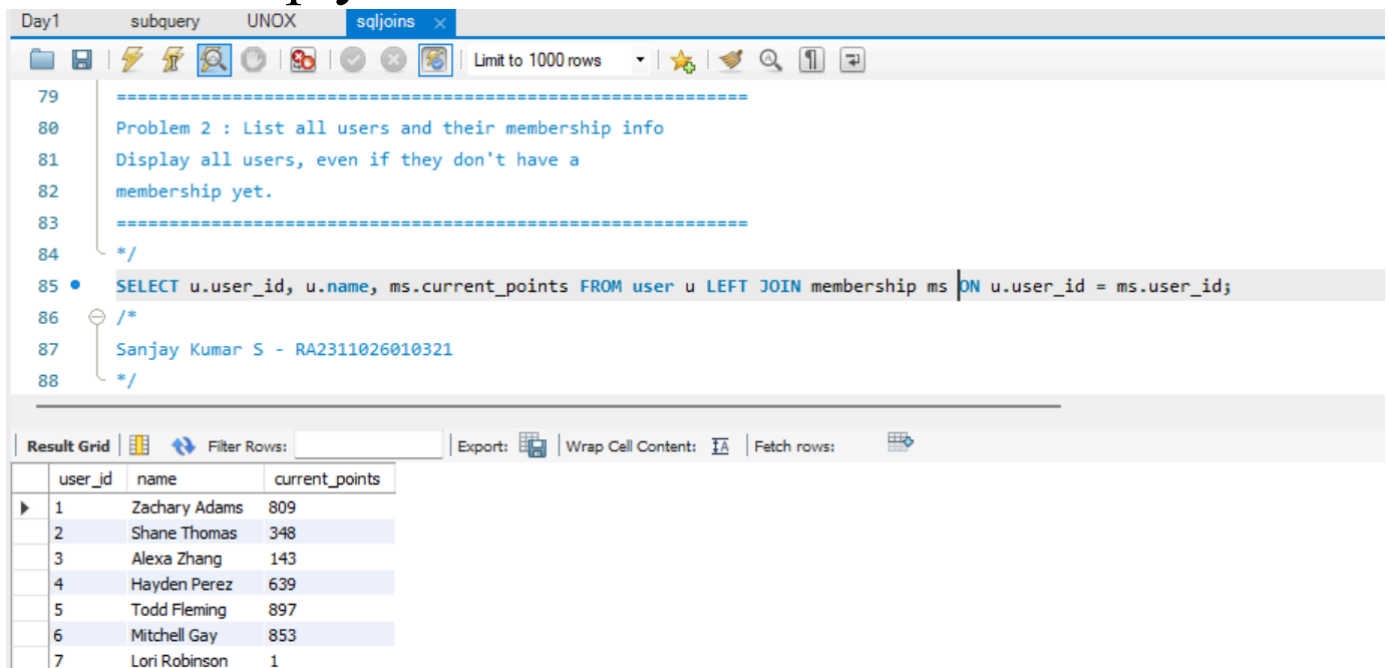
movie_id	title	genre	rating	status	poster_image_url
1	Avatar	Sci-Fi	5.1	Coming Soon	https://placekitten.com/410/120
2	Nine	Comedy	7.5	Now Showing	https://placekitten.com/80/766
3	Hard Luck	Horror	1.4	Now Showing	https://dummyimage.com/718x260
4	The Officers' Ward	Comedy	1.7	Coming Soon	https://dummyimage.com/884x721
5	You Are So Beautiful	Horror	5.9	Coming Soon	https://dummyimage.com/882x449
6	It's Complicated	Horror	9.0	Now Showing	https://picsum.photos/886/659
7	Sherlock Holmes	Drama	2.8	Coming Soon	https://placekitten.com/473/484

Problem 2: List all users and their membership info

Complexity: **Easy**

Scenario:

Display all users, even if they don't have a membership yet.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains a SQL query that uses a LEFT JOIN to list all users and their current points, even if they do not have a membership yet. The result grid displays the output of the query, showing 7 rows of data.

SQL Query:

```

79 =====
80 Problem 2 : List all users and their membership info
81 Display all users, even if they don't have a
82 membership yet.
83 =====
84 */
85 • SELECT u.user_id, u.name, ms.current_points FROM user u LEFT JOIN membership ms ON u.user_id = ms.user_id;
86 /*
87 Sanjay Kumar S - RA2311026010321
88 */

```

Result Grid:

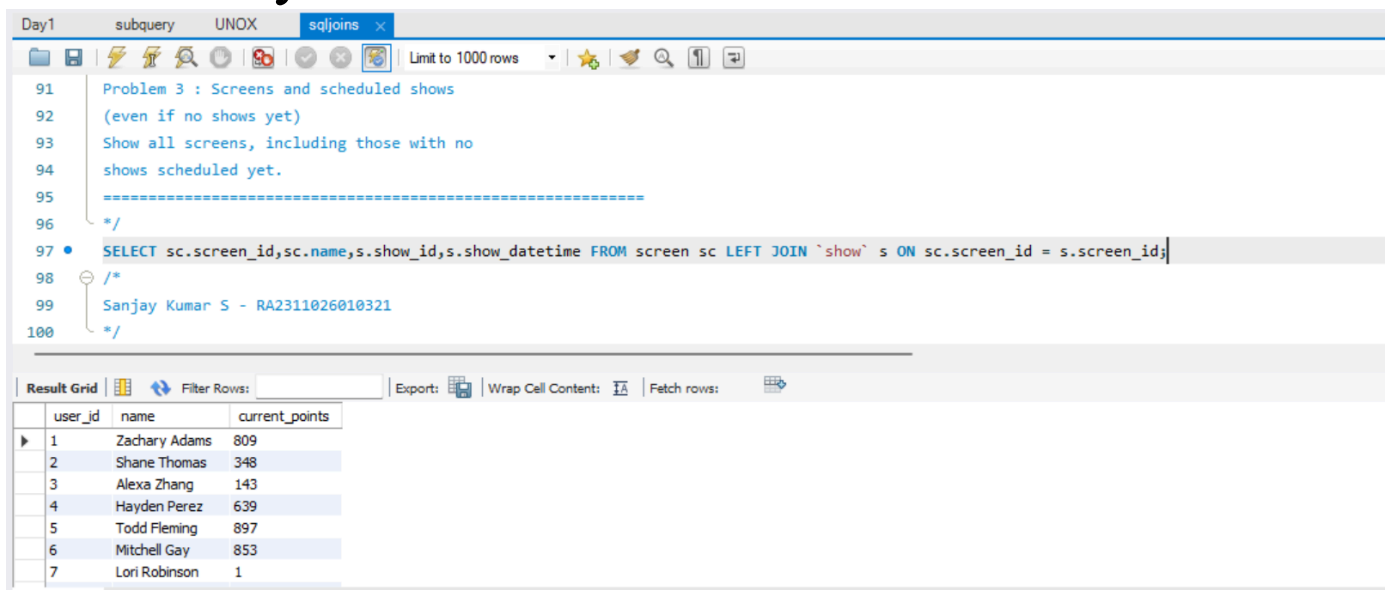
user_id	name	current_points
1	Zachary Adams	809
2	Shane Thomas	348
3	Alexa Zhang	143
4	Hayden Perez	639
5	Todd Fleming	897
6	Mitchell Gay	853
7	Lori Robinson	1

Problem 3 (Higher-order): Screens and scheduled shows (even if no shows yet)

Complexity: **Easy**

Scenario:

Show all screens, including those with no shows scheduled yet.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL query:

```

91 Problem 3 : Screens and scheduled shows
92 (even if no shows yet)
93 Show all screens, including those with no
94 shows scheduled yet.
95 =====
96 */
97 • SELECT sc.screen_id,sc.name,s.show_id,s.show_datetime FROM screen sc LEFT JOIN `show` s ON sc.screen_id = s.screen_id;
98 /*
99 Sanjay Kumar S - RA2311026010321
100 */

```

The result grid displays the following data:

user_id	name	current_points
1	Zachary Adams	809
2	Shane Thomas	348
3	Alexa Zhang	143
4	Hayden Perez	639
5	Todd Fleming	897
6	Mitchell Gay	853
7	Lori Robinson	1

Problem 4: Right JOIN Example (Screens and Shows)

Complexity: **Easy**

Scenario:

Display all shows and their associated screens.
Include shows even if screen info is missing.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is a Right JOIN between the 'screen' table (aliased as 'sc') and the 'show' table (aliased as 's'). The query is as follows:

```

104 (Screens and Shows)
105 Display all shows and their associated
106 screens. Include shows even if screen
107 information is missing.
108 =====
109 */
110 • SELECT sc.screen_id, sc.name,s.show_id,s.show_datetime FROM screen sc RIGHT JOIN `show` s ON sc.screen_id = s.screen_id;
111 /*
112 Sanjay Kumar S - RA2311026010321
113 */

```

The result grid shows the following data:

screen_id	name	show_id	show_datetime
2	B	1	2025-01-25 06:38:00
3	C	2	2025-03-20 05:48:00
4	D	3	2025-01-28 09:22:00
2	B	4	2025-02-28 21:35:00
1	A	5	2025-02-21 19:28:00
3	C	6	2025-01-23 17:52:00
4	D	7	2025-02-07 18:44:00

The result grid is labeled "Result 7" and has a close button (X).

LEVEL 3 – FULL OUTER JOIN & CROSS JOIN

Problem 1: All movies and all reviews

Complexity: **Hard**

Scenario:

Display all movies and reviews, including movies without reviews and reviews without movies (simulated FULL OUTER JOIN using UNION).

The screenshot shows a SQL IDE window with a query editor and a result grid. The query is as follows:

```

126  /*
127  • SELECT m.movie_id,m.title,r.review_id
128  FROM movie m
129  LEFT JOIN review r
130  ON m.movie_id = r.movie_id
131  UNION
132  SELECT m.movie_id,m.title, r.review_id FROM movie m RIGHT JOIN review r ON m.movie_id = r.movie_id;
133  /*
134  Sanjay Kumar S - RA2311026010321
135  */

```

The result grid shows the following data:

movie_id	title	review_id
1	Avatar	1
1	Avatar	2
1	Avatar	3
1	Avatar	4
1	Avatar	5
1	Avatar	6
1	Avatar	7

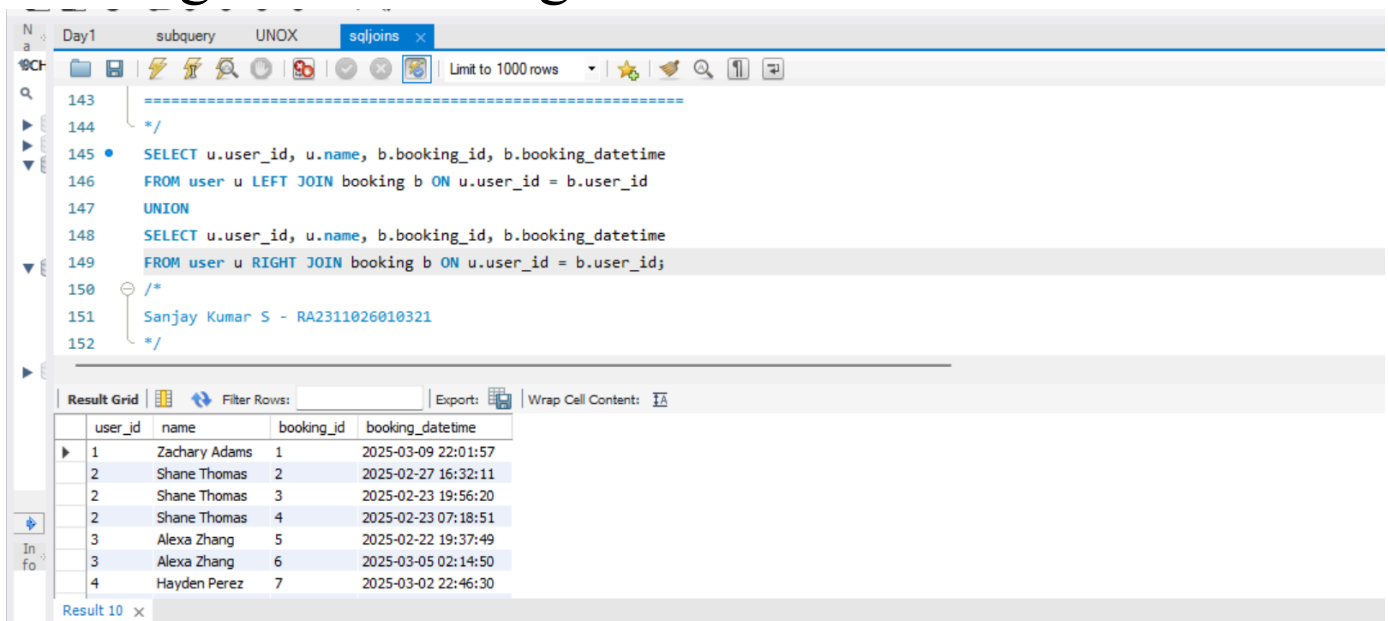
Result 8 x

Problem 2: Users and bookings (FULL OUTER JOIN simulation)

Complexity: **Hard**

Scenario:

List all users and bookings, including users with no bookings and bookings with no user info.



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is as follows:

```

143  =====
144  */
145  SELECT u.user_id, u.name, b.booking_id, b.booking_datetime
146  FROM user u LEFT JOIN booking b ON u.user_id = b.user_id
147  UNION
148  SELECT u.user_id, u.name, b.booking_id, b.booking_datetime
149  FROM user u RIGHT JOIN booking b ON u.user_id = b.user_id;
150  */
151  Sanjay Kumar S - RA2311026010321
152  */
  
```

The results grid displays the following data:

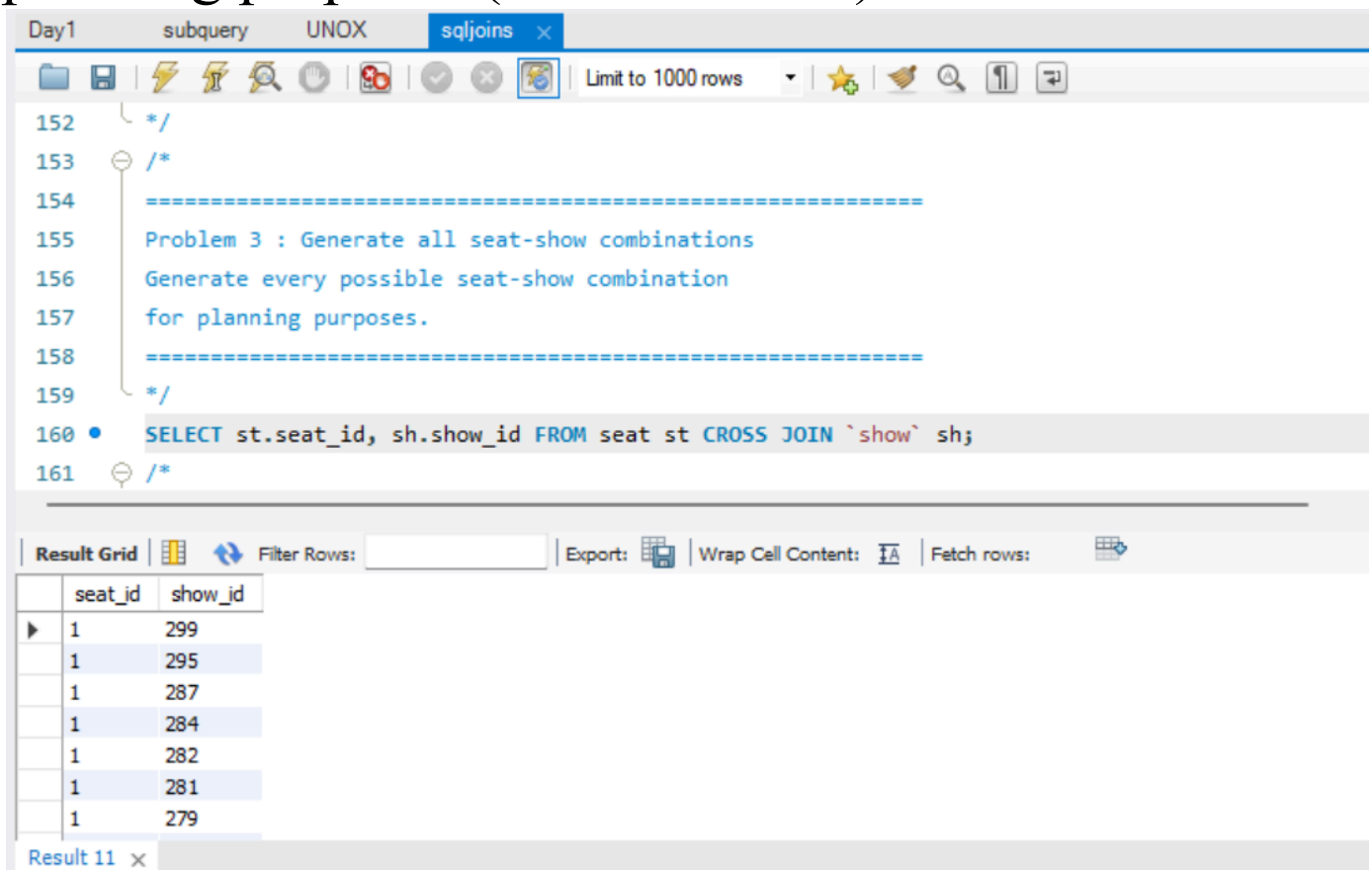
user_id	name	booking_id	booking_datetime
1	Zachary Adams	1	2025-03-09 22:01:57
2	Shane Thomas	2	2025-02-27 16:32:11
2	Shane Thomas	3	2025-02-23 19:56:20
2	Shane Thomas	4	2025-02-23 07:18:51
3	Alexa Zhang	5	2025-02-22 19:37:49
3	Alexa Zhang	6	2025-03-05 02:14:50
4	Hayden Perez	7	2025-03-02 22:46:30

Problem 3 (Higher-order): Generate all seat–show combinations

Complexity: **Easy**

Scenario:

Generate every possible seat–show combination for planning purposes (CROSS JOIN).



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains a SQL query that performs a CROSS JOIN between a 'seat' table and a 'show' table. The result grid displays the output of this query, showing all possible combinations of seat IDs and show IDs.

Query:

```

152  */
153  /*
154  =====
155  Problem 3 : Generate all seat-show combinations
156  Generate every possible seat-show combination
157  for planning purposes.
158  =====
159  */
160  • SELECT st.seat_id, sh.show_id FROM seat st CROSS JOIN `show` sh;
161  */

```

Result Grid:

seat_id	show_id
1	299
1	295
1	287
1	284
1	282
1	281
1	279

